Undecidability

CSE 211 (Theory of Computation)

Atif Hasan Rahman

Assistant Professor
Department of Computer Science and Engineering
Bangladesh University of Engineering & Technology

Adapted from slides by

Dr. Muhammad Masroor Ali

Quest to Decide All Mathematical Questions

- David Hilbert asked whether it was possible to find an algorithm for determining the truth or falsehood of any mathematical proposition
- In 1900, he identified 23 mathematical problems and posed them as a challenge for the coming century
- The tenth problem was to devise a process to determine whether a multivariate polynomial such as

$$6x^3yz^2 + 3xy^2 - x^3 - 10$$

  has integral (all variables) roots
- We now know, no algorithm exists for this task; it is algorithmically "unsolvable"

Quest to Decide All Mathematical Questions

- In 1931, Kurt Gödel published his famous *incompleteness theorem*
  - “no consistent system of axioms whose theorems can be listed by an effective procedure (i.e., an algorithm) is capable of proving all truths about the arithmetic of the natural numbers” (wiki)
- He constructed a formula in the predicate calculus (logic using quantified variables) applied to integers, which asserted that the formula itself could be neither proved nor disproved within the predicate calculus

Quest to Decide All Mathematical Questions

- In 1933, Austrian-American mathematician Gödel, with Jacques Herbrand, created a formal definition of a class called general recursive functions
- In 1936, Alonzo Church created a method for defining functions called the λ-calculus
- Also in 1936, before learning of Church's work, Alan Turing created a theoretical model for machines, the Turing machines
- Church and Turing proved that these three formally defined classes of computable functions coincide: a function is λ-computable if and only if it is Turing computable if and only if it is general recursive

Church Turing thesis

- The unprovable assumption that any general way to compute will allow us to compute only the general recursive functions is called the ***Church Turing thesis***
- Unprovable since “general way to compute” is an informal term but we have already seen equivalence of TM and computers
- In 1970, Yuri Matijasevic, building on the work of Martin Davis, Hilary Putnam, and Julia Robinson, showed that no algorithm exists for testing whether a multivariate polynomial has integral roots

Some Undecidable Problems

- Hilbert's *Entscheidungsproblem* - asking for an algorithm to decide whether a given statement is provable from the axioms using the rules of logic
- Satisfiability of first order Horn clauses
- The halting problem - determining whether a Turing machine halts on a given input
- Determining if a context-free grammar is ambiguous
- Determining whether two context-free grammars generate the same language
- The mortal matrix problem: determining, given a finite set of $n \times n$ matrices with integer entries, whether they can be multiplied in some order, possibly with repetition, to yield the zero matrix.
  - known to be undecidable for a set of six or more 3×3 matrices, or a set of two 15×15 matrices

RE Languages

- A language L is recursively enumerable (abbreviated RE) if $L = L(M)$ for some TM M
- We'll divide the recursively enumerable (RE languages) into two classes:
  i. One class, which corresponds to what we commonly think of as an algorithm, has a TM that not only recognizes the language, but it also halts eventually, regardless of whether or not it reaches an accepting state
  ii. The second class of languages consists of those RE languages that are not accepted by any Turing machine with the guarantee of halting. If the input is in the language, we'll eventually know that, but if the input is not in the language, then the TM may run forever

Recursive Languages

- We call a language L recursive if $L = L(M)$ for some Turing machine M such that:
  i. If w is in L, then M accepts (and therefore halts).
  ii. If w is not in L, then M eventually halts, although it doesn't enter an accepting state.
- A problem L is called *decidable* if it is a recursive language, and it is called *undecidable* if it is not a recursive language.

Relationship between the recursive, RE, and non-RE languages

- Three classes of languages:
  i. The recursive languages
  ii. The languages that are recursively enumerable but not recursive.
  iii. The non-recursively-enumerable (non-RE) languages.

Relationship between the recursive, RE, and non-RE languages

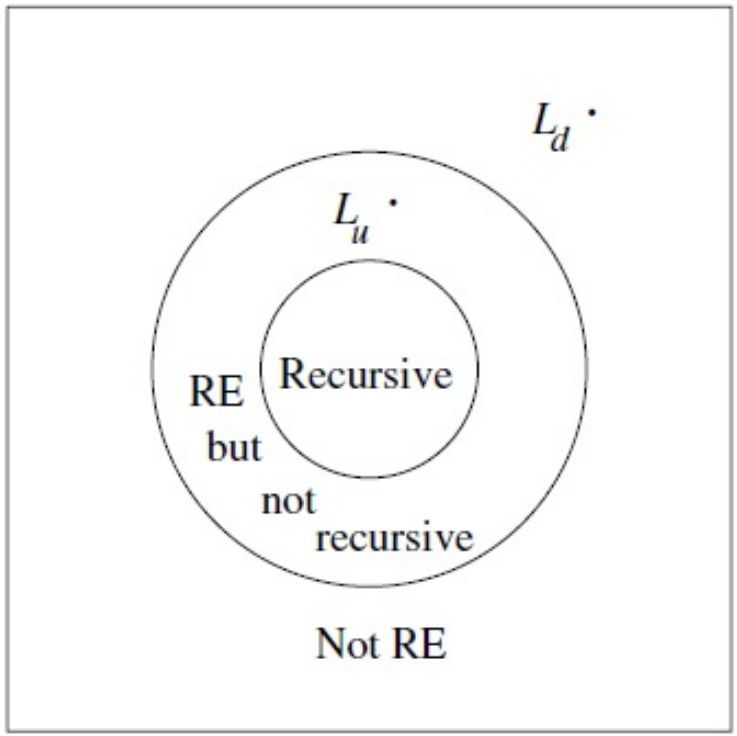


Figure 9.2: Relationship between the recursive, RE, and non-RE languages

Some Decidable Languages

Concerning Regular Languages

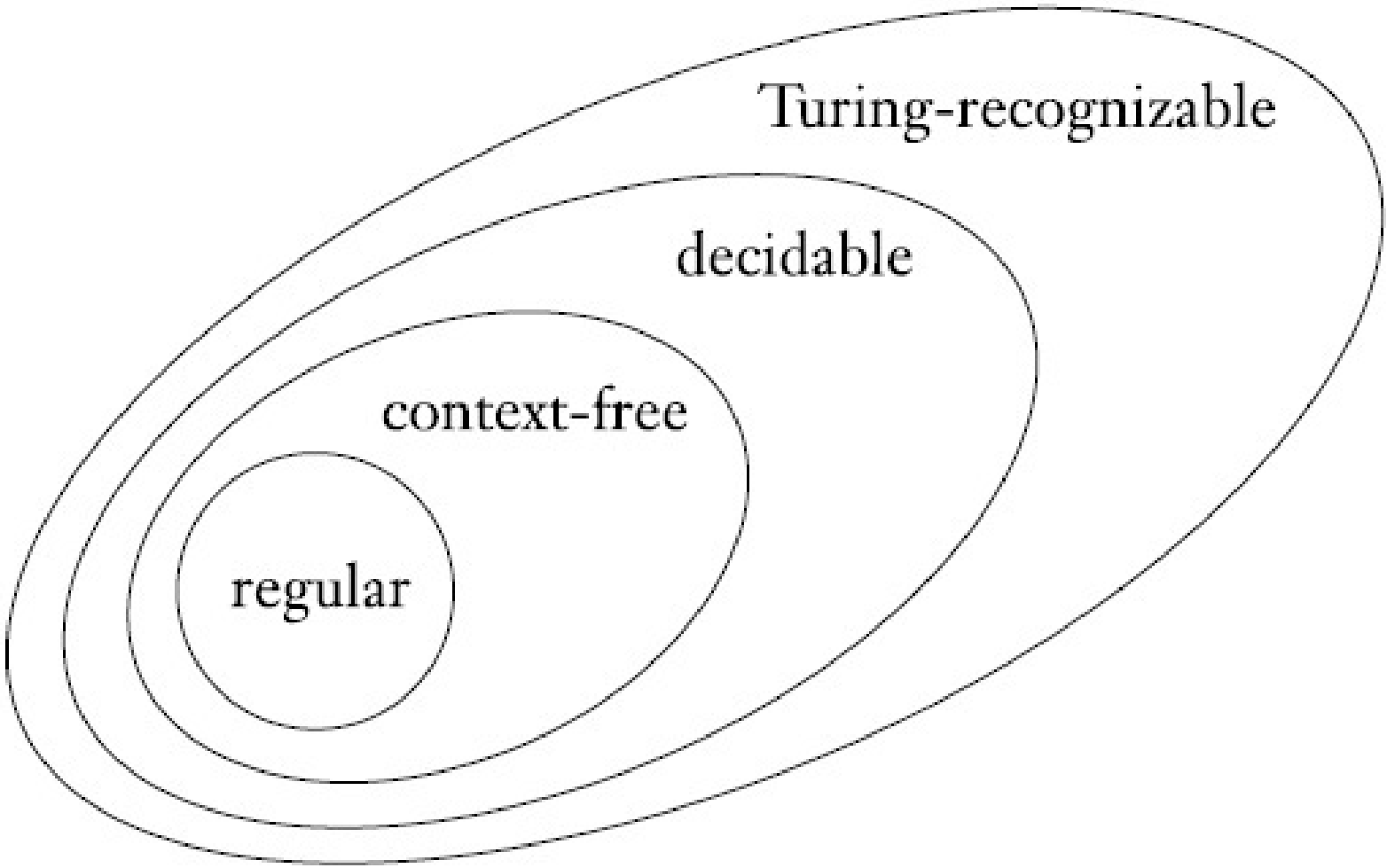
- $A_{DFA} = \{< B, w > | B \text{ is a DFA that accepts input string } w\}$
- $A_{NFA} = \{< B, w > | B \text{ is an NFA that accepts input string } w\}$
- $A_{REX} = \{< R, w > | R \text{ is a regular expression that generates } w\}$
- $E_{DFA} = \{< A > | A \text{ is a DFA and } L(A) = \emptyset\}$
- $EQ_{DFA} = \{< A, B > | A, B \text{ are DFAs and } L(A) = L(B)\}$

Some Decidable Languages

Concerning Context-Free Languages

- $A_{CFG} = \{< G, w > | G \text{ is a CFG that generates } w\}$
- $E_{CFG} = \{< G > | G \text{ is a CFG and } L(G) = \emptyset\}$
- Recall that
  $EQ_{CFG} = \{< G, H > | G, H \text{ are CFGs and } L(G) = L(G)\}$ is undecidable

Relationship among classes of languages



A Language That Is Not Recursively Enumerable

- Our goal is to prove undecidable the language consisting of pairs (M, w) such that:
  1. M is a Turing machine (suitably coded, in binary) with input alphabet $\{0, 1\}$,
  2. w is a string of 0's and 1's, and
  3. M accepts input.
- If this problem with inputs restricted to the binary alphabet is undecidable, then surely the more general problem, where TM's may have any alphabet, is undecidable.

A Language That Is Not Recursively Enumerable

- Our goal is to prove undecidable the language consisting of pairs (M, w) such that:
  1. M is a Turing machine (suitably coded, in binary) with input alphabet $\{0, 1\}$,
  2. w is a string of 0's and 1's, and
  3. M accepts input.
- If this problem with inputs restricted to the binary alphabet is undecidable, then surely the more general problem, where TM's may have any alphabet, is undecidable.

A Language That . . . Enumerable

- It involves the language L_d , the "diagonalization language," which consists of all those strings w such that the TM represented by w does not accept the input w.
- We shall show that L_d has no Turing machine at all that accepts it.
- Remember that showing there is no Turing machine at all for a language is showing something stronger than that the language is undecidable (i.e., that it has no algorithm, or TM that, always halts).

Enumerating the Binary Strings

- We shall need to assign integers to all the binary strings so that each string corresponds to one integer.
- Each integer corresponds to one string.
- If w is a binary string, treat $1w$ as a binary integer i.
- Then we shall call w the ith string.
- That is, ϵ is the first string, 0 is the second, 1 the third, 00 the fourth, 01 the fifth, and so on.
- Equivalently, strings are ordered by length, and strings of equal length are ordered lexicographically.
- Hereafter, we shall refer to the ith string as w_i.

Codes for Turing Machines

- We want to devise a binary code for Turing machines so that each TM with input alphabet $\{0, 1\}$ may be thought of as a binary string.
- We know how to enumerate the binary strings, we shall then have an identification of the Turing machines with the integers.
- We can talk about "the ith Turing machine, M_i".

Codes for Turing Machines

- To represent a TM $M = (Q, \{0, 1\}, \Gamma, \delta\ , q_1, B, F)$ as a binary string, we must first assign integers to the states, tape symbols, and directions L and R.

- We shall assume the states are $q_1, q_2, \ldots, q_r$ for some r.
- The start state will always be q_1, and q_2 will be the only accepting state.
- Since we may assume the TM halts whenever it enters an accepting state, there is never any need for more than one accepting state.

Codes for Turing Machines

- To represent a TM $M = (Q, \{0, 1\}, \Gamma, \delta\ , q_1, B, F)$ as a binary string, we must first assign integers to the states, tape symbols, and directions L and R.

- We shall assume the tape symbols are $X_1, X_2, \ldots, X_s$ for some s.
- X_1 always will be the symbol 0, X_2 will be 1, and X_3 will be B, the blank.
- However, other tape symbols can be assigned to the remaining integers arbitrarily.

Codes for Turing Machines

- To represent a TM $M = (Q, \{0, 1\}, \Gamma, \delta\ , q_1, B, F)$ as a binary string, we must first assign integers to the states, tape symbols, and directions L and R.

- We shall refer to direction L as D_1 and direction R as D_2.

Codes for Turing Machines

- Each TM M can have integers assigned to its states and tape symbols in many different orders.
- There will be more than one encoding of the typical TM.
- However, that fact is unimportant in what follows.
- We shall show that no encoding can represent a TM M such that $L(M) = L_d$.

Codes for Turing Machines

- Once we have established an integer to represent each state, symbol, and direction, we can encode the transition function δ .
- Suppose one transition rule is $\delta(q_i, X_j) = (q_k, X_l, D_m)$, for some integers i, j, k, l, and m.
- We shall code this rule by the string $0^i 1 0^j 1 0^k 1 0^l 1 0^m$.
- Notice that, since all of i, j, k, l, and m are at least one, there are no occurrences of two or more consecutive 1's within the code for a single transition.

Codes for Turing Machines

- A code for the entire TM M consists of all the codes for the transitions, in some order, separated by pairs of 1's:

$$c_1 11 c_2 11 \cdots c_{n-1} 11 C_n$$

where each of the C's is the code for one transition of M.

Example

- Let the TM in question be

$$M = (\{q_1, q_2, q_3\}, \{0, 1\}, \{0, 1, B\}, \delta\ , q_1, B, \{q_2\})$$

- δ consists of the rules:

$$\begin{aligned}
\delta\ (q_1, 1) &= (q_3, 0, R) \\
\delta\ (q_3, 0) &= (q_1, 1, R) \\
\delta\ (q_3, 1) &= (q_2, 0, R) \\
\delta\ (q_3, B) &= (q_3, 1, L)
\end{aligned}$$

Example-continued

$M = (\{q_1, q_2, q_3\}, \{0, 1\}, \{0, 1, B\}, \delta, q_1, B, \{q_2\})$

- We shall assume the states are $q_1, q_2, \ldots, q_r$ for some r.
- The start state will always be q_1, and q_2 will be the only accepting state.

- $q_1 = 0^1$
- $q_2 = 0^2$
- $q_3 = 0^3$

Example-continued

$M = (\{q_1, q_2, q_3\}, \{0, 1\}, \{0, 1, B\}, \delta, q_1, B, \{q_2\})$

- We shall assume the tape symbols are $X_1, X_2, \ldots, X_s$ for some s.
- X_1 always will be the symbol 0, X_2 will be 1, and X_3 will be B, the blank.
- However, other tape symbols can be assigned to the remaining integers arbitrarily.

- $0 = X_1 = 0^1$
- $1 = X_2 = 0^2$
- $B = X_3 = 0^3$

Example-continued

$M = (\{q_1, q_2, q_3\}, \{0, 1\}, \{0, 1, B\}, \delta, q_1, B, \{q_2\})$

- We shall refer to direction L as D_1 and direction R as D_2.

- $L = D_1 = 0^1$
- $R = D_2 = 0^2$

Example-continued

$M = (\{q_1, q_2, q_3\}, \{0, 1\}, \{0, 1, B\}, \delta, q_1, B, \{q_2\})$

$q_1 = 0^1$	$0 = X_1 = 0^1$	
$q_2 = 0^2$	$1 = X_2 = 0^2$	$L = D_1 = 0^1$
$q_3 = 0^3$	$B = X_3 = 0^3$	$R = D_2 = 0^2$

$$\delta(q_1, 1) = (q_3, 0, R) \qquad \delta(q_3, 1) = (q_2, 0, R)$$
$$\delta(q_3, 0) = (q_1, 1, R) \qquad \delta(q_3, B) = (q_3, 1, L)$$

$0^1 1 0^2 1 0^3 1 0^1 1 0^2 = 0100100010100$

0001010100100

$0001001001 0100$

0001000100010010

Example-continued

$M = (\{q_1, q_2, q_3\}, \{0, 1\}, \{0, 1, B\}, \delta, q_1, B, \{q_2\})$

$q_1 = 0^1$	$0 = X_1 = 0^1$	
$q_2 = 0^2$	$1 = X_2 = 0^2$	$L = D_1 = 0^1$
$q_3 = 0^3$	$B = X_3 = 0^3$	$R = D_2 = 0^2$

$\delta(q_1, 1)$	$=$	$(q_3, 0, R)$	$\delta(q_3, 1)$	$=$	$(q_2, 0, R)$
$\delta(q_3, 0)$	$=$	$(q_1, 1, R)$	$\delta(q_3, B)$	$=$	$(q_3, 1, L)$

$0^1 1 0^2 1 0^3 1 0^1 1 0^2 = 0100100010100$

0001010100100

0001001000100100

0001000100010010

Example-continued

$M = (\{q_1, q_2, q_3\}, \{0, 1\}, \{0, 1, B\}, \delta, q_1, B, \{q_2\})$

$q_1 = 0^1$	$0 = X_1 = 0^1$	
$q_2 = 0^2$	$1 = X_2 = 0^2$	$L = D_1 = 0^1$
$q_3 = 0^3$	$B = X_3 = 0^3$	$R = D_2 = 0^2$

$$\delta(q_1, 1) = (q_3, 0, R) \qquad \delta(q_3, 1) = (q_2, 0, R)$$
$$\delta(q_3, 0) = (q_1, 1, R) \qquad \delta(q_3, B) = (q_3, 1, L)$$

$0^1 1 0^2 1 0^3 1 0^1 1 0^2 = 0100100010100$
0001010100100
00010010010100
0001000100010010

Example-continued

$M = (\{q_1, q_2, q_3\}, \{0, 1\}, \{0, 1, B\}, \delta, q_1, B, \{q_2\})$

$q_1 = 0^1$	$0 = X_1 = 0^1$	
$q_2 = 0^2$	$1 = X_2 = 0^2$	$L = D_1 = 0^1$
$q_3 = 0^3$	$B = X_3 = 0^3$	$R = D_2 = 0^2$

$$\delta(q_1, 1) = (q_3, 0, R) \qquad \delta(q_3, 1) = (q_2, 0, R)$$
$$\delta(q_3, 0) = (q_1, 1, R) \qquad \delta(q_3, B) = (q_3, 1, L)$$

$0^1 1 0^2 1 0^3 1 0^1 1 0^2 = 0100100010100$
0001010100100
00010010010100
0001000100010010

Example-continued

- A code for the entire TM M consists of all the codes for the transitions, in some order, separated by pairs of 1's:

$$c_1 11 c_2 11 \cdots c_{n-1} 11 C_n$$

where each of the C's is the code for one transition of M.

0100100010100
0001010100100
00010010010100
0001000100010010

A code for M is

0100100010100**11**0001010100100**11**0001001001010**11**0001000100010010

Example-continued

- A code for the entire TM M consists of all the codes for the transitions, in some order, separated by pairs of 1's:

$$c_1 11 c_2 11 \cdots c_{n-1} 11 C_n$$

where each of the C's is the code for one transition of M.

0100100010100
0001010100100
0001001001010 0
0001000100010010

A code for M is

0100100010100110001010100100110001001001010011000100010010010

Example-continued

- A code for the entire TM M consists of all the codes for the transitions, in some order, separated by pairs of 1's:

$$c_1 11 c_2 11 \cdots c_{n-1} 11 C_n$$

where each of the C's is the code for one transition of M.

0100100010100
0001010100100
00010010010100
0001000100010010

A code for M is

0100100010100110001010100100110001001001010011000100010010010

Example-continued

- A code for the entire TM M consists of all the codes for the transitions, in some order, separated by pairs of 1's:

$$c_1 11 c_2 11 \cdots c_{n-1} 11 C_n$$

where each of the C's is the code for one transition of M.

0100100010100
0001010100100
00010010010100
0001000100010010

A code for M is

0100100010100110001010100100110001001001010011000100010010

Example-continued

- A code for the entire TM M consists of all the codes for the transitions, in some order, separated by pairs of 1's:

$$c_1 11 c_2 11 \cdots c_{n-1} 11 C_n$$

where each of the C's is the code for one transition of M.

0100100010100
0001010100100
00010010010100
0001000100010010

A code for M is

0100100010100110001010100100110001001001010011000100010001001 0

Example-continued

- A code for the entire TM M consists of all the codes for the transitions, in some order, separated by pairs of 1's:

$$c_1 11 c_2 11 \cdots c_{n-1} 11 C_n$$

where each of the C's is the code for one transition of M.

0100100010100
0001010100100
00010010010100
0001000100010010

A code for M is

0100100010100110001010100100110001001001010011000100010010

Example-continued

- Note that there are many other possible codes for M.
- In particular, the codes for the four transitions may be listed in any of 4! orders, giving us 24 codes for M.

The Diagonalization Language

- M_i, the "ith Turing machine" is that TM M whose code is w_i, the ith binary string.
- Many integers do not correspond to any TM at all.
- For instance, 11001 does not begin with 0.
- 0010111010010100 is not valid because it has three consecutive 1's.

The Diagonalization Language

- If w_i is not a valid TM code, we shall take M_i to be the TM with one state and no transitions.
- That is, for these values of i, M_i is a Turing machine that immediately halts on any input.
- Thus, $L(M_i)$ is $\emptyset$ if w_i fails to be a valid TM code.

The Diagonalization Language

The language L_d , the diagonalization language, is the set of strings w_i such that w_i is not in $L(M_i)$.

- That is, L_d consists of all strings w such that the TM M whose code is w does not accept when given w as input.

The Diagonalization Language

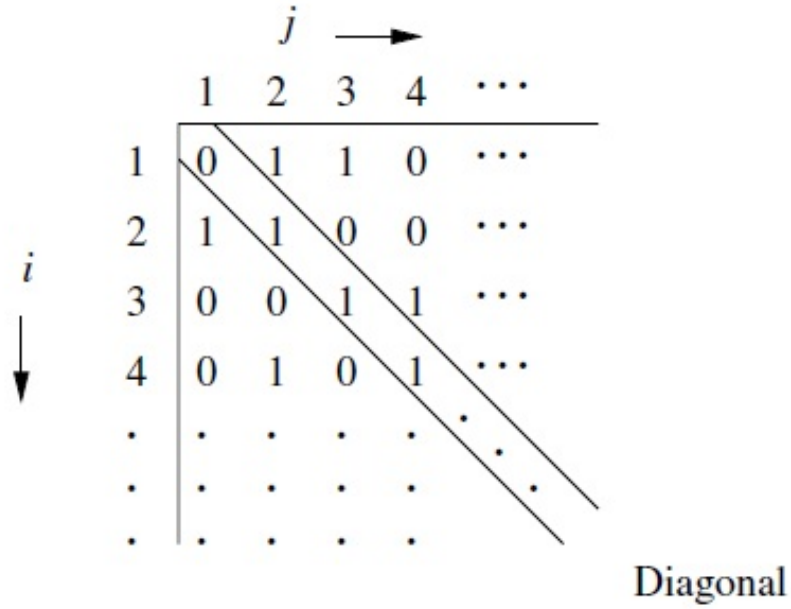
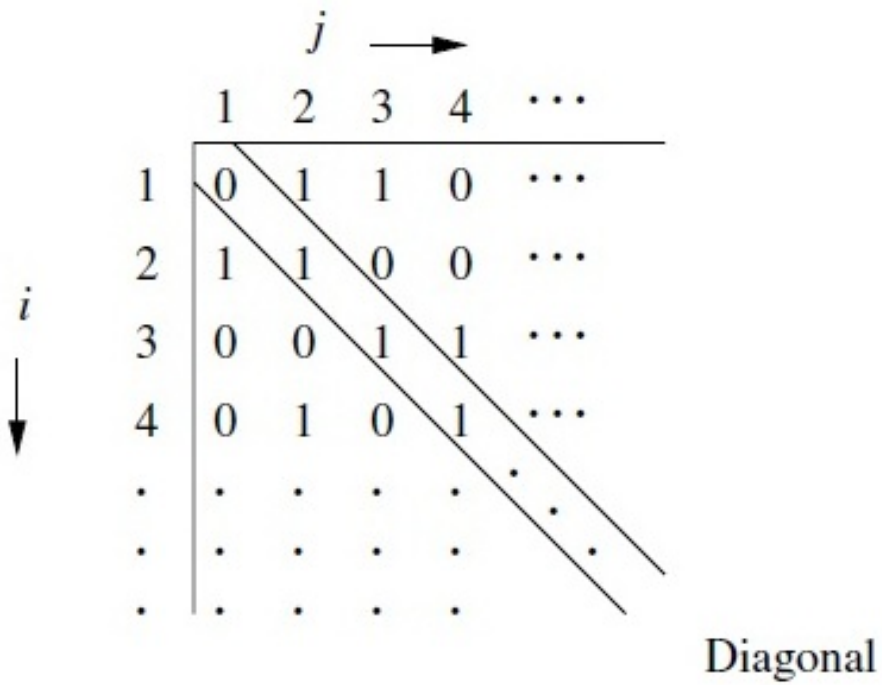


Figure 9.1: The table that represents acceptance of strings by Turing machines

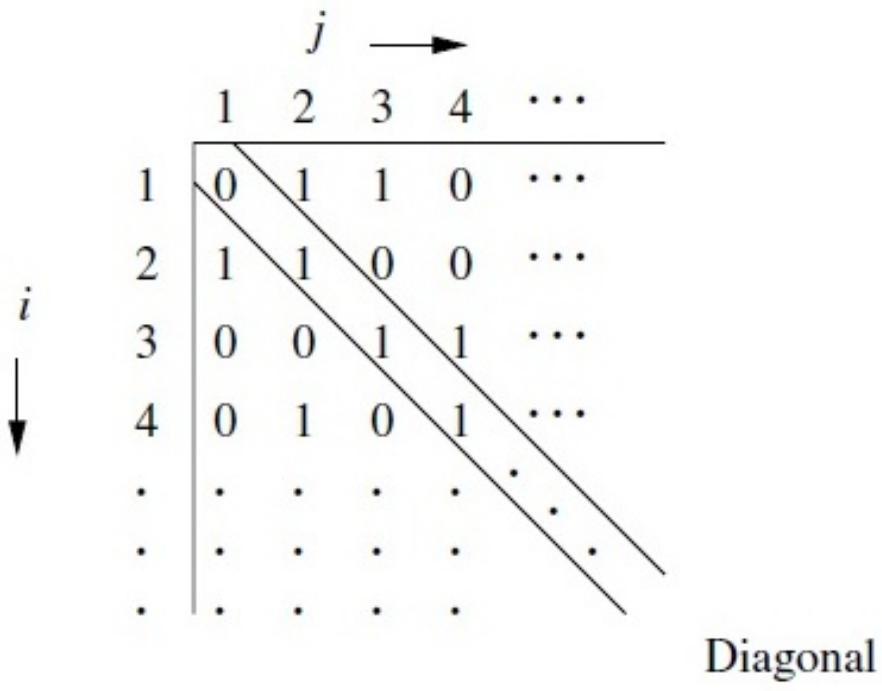
- The reason L_d is called a "diagonalization" language can be seen if we consider the figure.

The Diagonalization Language



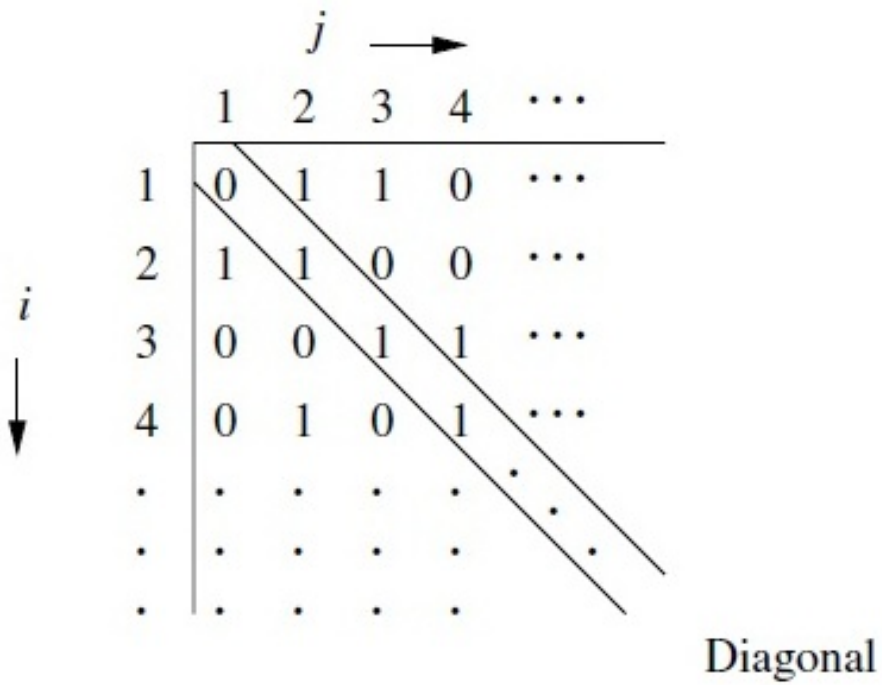
- This table tells for all i and j, whether the TM M_i accepts input string w_j.
- 1 means "yes it does" and 0 means "no it doesn't."
- We may think of the ith row as the characteristic vector for the language $L(M_i)$.
- That is, the 1's in this row indicate the strings that are members of this language.

The Diagonalization Language



- The diagonal values tell whether M_i accepts w_i.
- To construct L_d , we complement the diagonal.
- In the correct table the complemented diagonal would begin $1, 0, 0, 0, \ldots$.
- Thus, L_d would contain $w_1 = \epsilon$, not contain w_2 through w_4, which are 0, 1, and 00, and so on.

The Diagonalization Language



- The trick of complementing the diagonal to construct the characteristic vector of a language that cannot be the language that appears in any row, is called *diagonalization*.
- It works because the complement of the diagonal is itself a characteristic vector describing membership in some language, namely L_d .

The Barber of Seville

The barber of Seville is that inhabitant of Seville who shaves every man in Seville that does not shave himself

- This is a paradox, because who will shave the barber of Seville himself?
- If the barber does not shave himself, he — according to definition — must shave himself, because he shaves all who do not shave themselves.
- If the barber does shave himself, he — according to definition — can not shave himself, because the barber only shaves those who do not shave themselves.

Proof that L_d is not Recursively Enumerable

- Following the intuition about characteristic vectors and the diagonal, we shall now prove formally a fundamental result about Turing machines.
- There is no Turing machine that accepts the language L_d .

Proof that L_d . . .

Theorem

L_d is not a recursively enumerable language. That is, there is no Turing machine that accepts L_d .

PROOF: Suppose L_d were $L(M)$ for some TM M.

- Since L_d is a language over alphabet $\{0, 1\}$, M would be in the list of Turing machines we have constructed, since it includes all TM's with input alphabet $\{0, 1\}$.
- Thus, there is at least one code for M, say i; that is, $M = M_i$.

Proof that L_d . . .

Now, ask if w_i is in L_d .

- If w_i is in L_d , then M_i accepts w_i.
  - But then, by definition of L_d , w_i is not in L_d , because L_d contains only those w_j such that M_j does not accept w_j.
- Similarly, if w_i is not in L_d , then M_i does not accept w_i.
  - Thus, by definition of L_d , w_i is in L_d .

Proof that L_d . . .

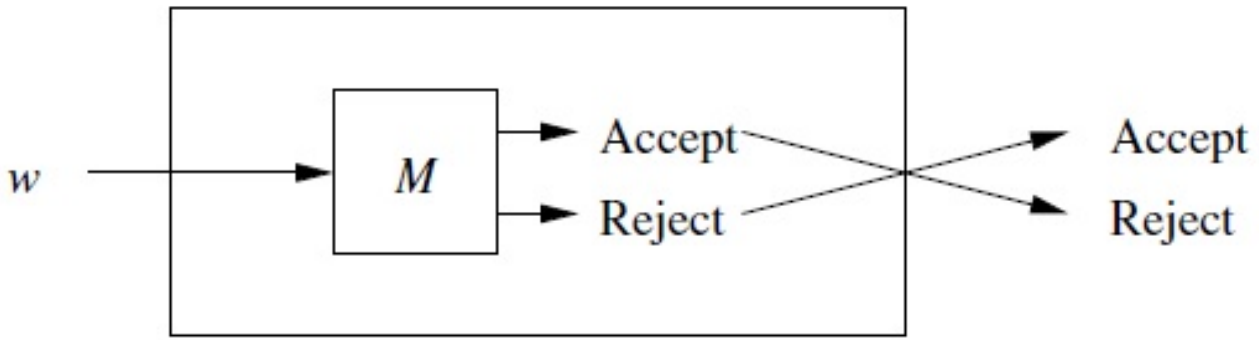
- Since w_i can neither be in L_d nor fail to be in L_d , we conclude that there is a contradiction of our assumption that M exists.
- That is, L_d is not a recursively enumerable language.

Complements of Recursive and RE languages

Theorem

If L is a recursive language, so is $\overline{L}$.

PROOF: Suppose $L = L(M)$ for some TM M. We construct a TM M such that $\overline{L} = L(\overline{M})$



Complements of Recursive and RE languages

M is modified as follows to create $\overline{M}$

- The accepting states of M are made nonaccepting states of $\overline{M}$ with no transitions i.e., in these states $\overline{M}$ will halt without accepting
- $\overline{M}$ has a new accepting state r; there are no transitions from r
- For each combination of a nonaccepting state of M and a tape symbol of M such that M has no transition i.e., M halts without accepting, add a transition to the accepting state r

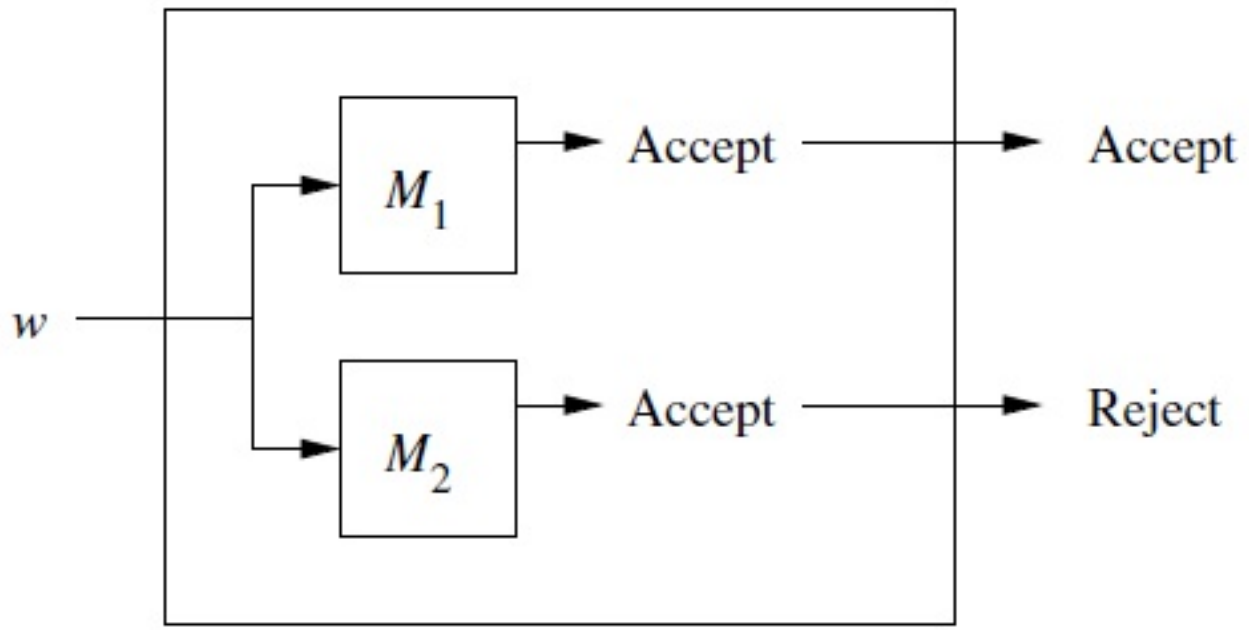
Since M is guaranteed to halt, we know that $\overline{M}$ is also guaranteed to halt. Moreover $\overline{M}$ accepts exactly those strings that M does not accept. Thus $\overline{M}$ accepts $\overline{L}$

Complements of Recursive and RE languages

Theorem

If both a language L and its complement are RE, then L is recursive. Note that $\overline{L}$ is recursive as well.

PROOF: Let $L = L(M_1)$ and $\overline{L} = L(M_2)$. Both M_1 and M_2 are simulated in parallel by a TM M



Complements of Recursive and RE languages

- We can make M a two tape TM, and then convert it to a one tape TM, to make the simulation easy and obvious.
- One tape of M simulates the tape of M_1 while the other tape of M simulates the tape of M_2.
- The states of M_1 and M_2 are each components of the state of M

Complements of Recursive and RE languages

- If input w to M is in L, then M_1 will eventually accept.
- If so, M accepts and halts.
- If w is not in L, then it is in $\overline{L}$, so M_2 will eventually accept.
- When M_2 accepts, M halts without accepting.

Thus, on all inputs M halts, and $L(M)$ is exactly L. Since M always halts, and $L(M) = L$, we conclude that L is recursive

What about $\overline{L_d}$?

- Since L_d is not RE, $\overline{L_d}$ cannot be recursive
- $\overline{L_d}$ could be either non-RE or RE-but-not-recursive
- It is in fact the latter

Universal Language

- The ***universal language***, L_u is the set of binary strings that encode, a pair (M, w) where
  - M is a TM with the binary input alphabet
  - w is a string in $(0 + 1)^*$
  - w is in $L(M)$
- That is, L_u is the set of strings representing a TM and an input accepted by that TM

Universal TM

- We shall show that there is a TM U, often called the ***universal Turing machine***, such that $L_u = L(U)$
- We describe U as a multitape Turing machine
  - The transitions of M are stored initially on the first tape, along with the string w separated by 111
  - A second tape will be used to hold the simulated tape of M, using the same format as for the code of M. That is, tape symbol X_i of M will be represented by 0^i, and tape symbols will be separated by single 1s.
  - The third tape of U holds the state of M, with state q_i represented by i 0s

Universal TM

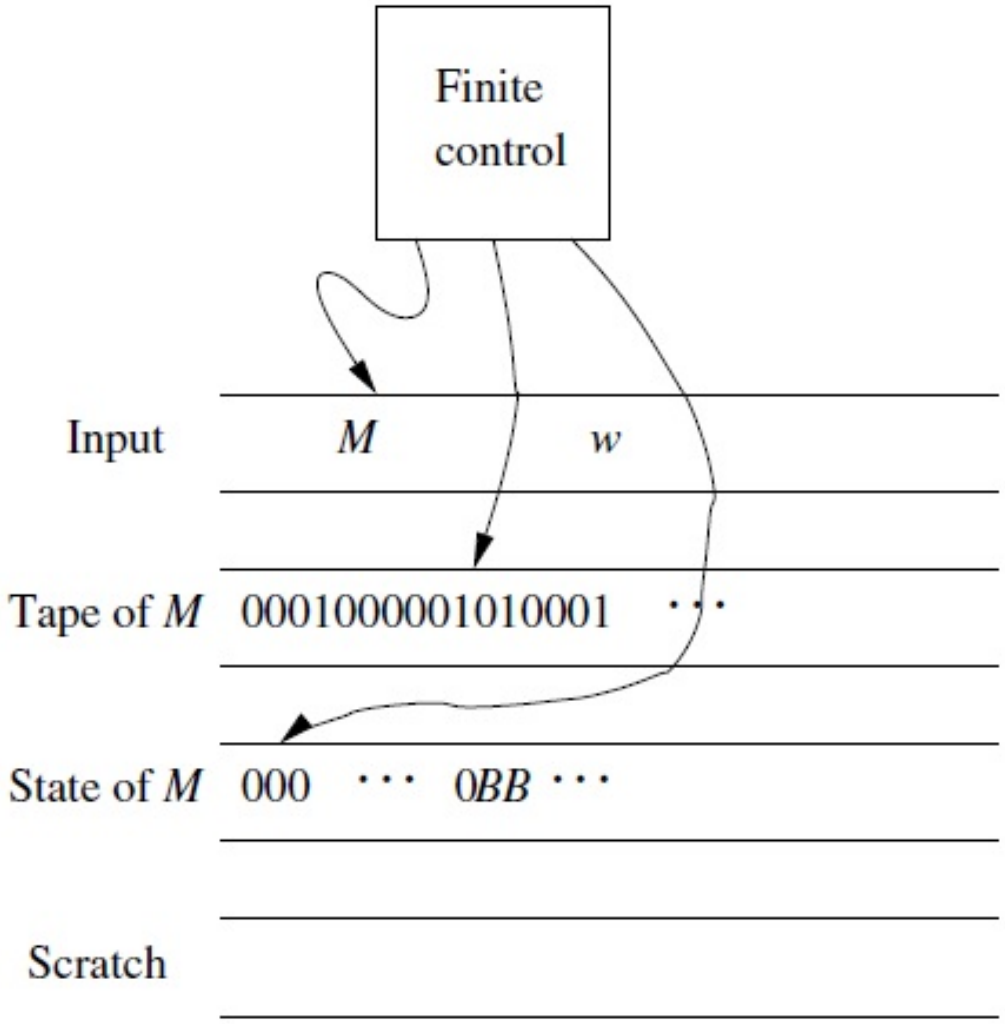


Figure 9.5: Organization of a universal Turing machine

Universal TM

The operation of U can be summarized as follows

1. Examine the input to make sure that the code for M is a legitimate code for some TM. If not, U halts without accepting
2. Initialize the second tape to contain the input w in its encoded form
3. Place 0, the start state of M, on the third tape, and move the head of U's second tape to the first simulated cell

Universal TM

4. To simulate a move of M, U searches on its first tape for a transition $0^i10^j10^k10^l10^m$, such that 0^i is the state on tape 3 and 0^j is the tape symbol of M that begins at the position on tape scanned by U. U should:
   - Change the contents of tape 3 to 0^k that is, simulate the state change of M
   - Replace 0^j on tape 2 by 0^l, that is, change the tape symbol of M. If more or less space is needed, use the scratch tape and the shifting over technique
   - Move the head on tape 2 to the position of the next 1 to the left or right, respectively depending on whether $m = 1$ (move left) or $m = 2$ (move right)

Universal TM

5. If M has no transition that matches the simulated state and tape symbol, then in (4) no transition will be found. Thus, M halts in the simulated configuration, and U must do likewise
6. If M enters its accepting state, then U accepts

In this manner, U simulates M on w. U accepts the coded pair (M, w) if and only if M accepts w

Universal Language

Theorem

L_u is RE but not recursive.

PROOF:

- We just proved that L_u is RE
- Suppose L_u were recursive, then the complement of L_u would also be recursive.
- However, if we have a TM M to accept $\overline{L_u}$, then we can construct a TM to accept L_d
- Since we already know that L_d is not RE, we have a contradiction of our assumption that L_u is recursive

Universal Language

Suppose $L(M) = \overline{L_u}$. As suggested below, we can modify TM M into a TM M' that accepts L_d

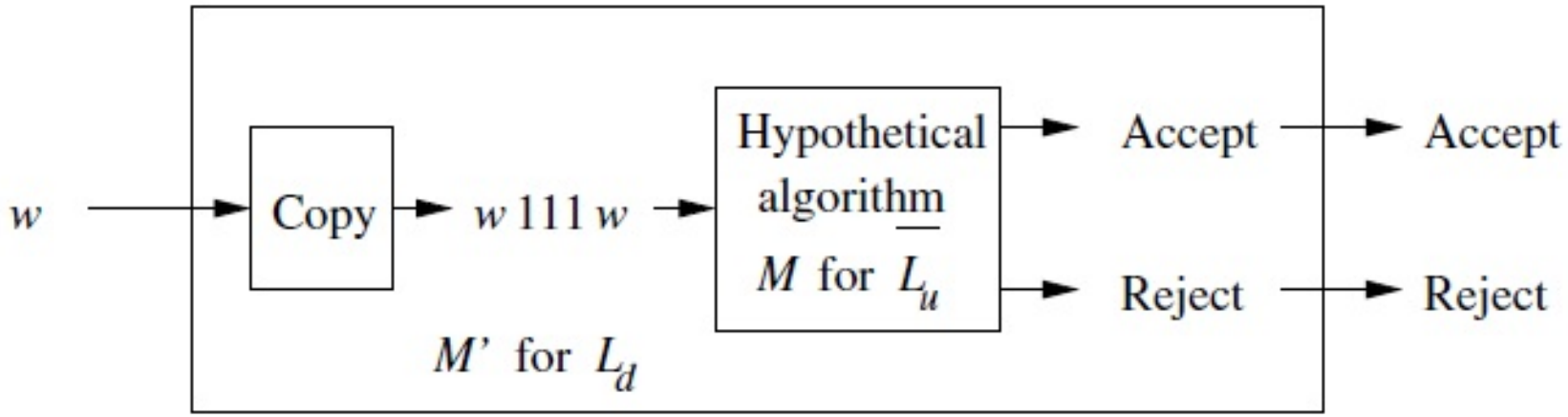


Figure 9.6: Reduction of L_d to $\overline{L_u}$

Universal Language

1. Given string w on its input, M' changes the input to $w111w$
2. M' simulates M on the new input. If w is w_i in our enumeration, then M' determines whether M_i accepts w_i. Since M accepts $\overline{L_u}$, it will accept if and only if M_i does not accept w_i i.e. w_i is in L_d

Thus, M' accepts w if and only if w is in L_d. Since we know M' cannot exist, we conclude that L_u is not recursive.

Halting Problem

- The *halting problem* for Turing machines is a problem similar to L_u - one that is RE but not recursive
- The original Turing machine of Turing accepted by halting, not by final state
- We could define $H(M)$ for TM M to be the set of inputs w such that M halts given input w, regardless of whether or not M accepts w
- Then, the halting problem is the set of pairs (M, w) such that w is in $H(M)$.
- This problem/language is another example of one that is RE but not recursive

Halting Problem is RE

- We can construct a Turing machine U_H similar to the *universal Turing machine*
- U_H takes as input a Turing machine M and a binary string w
- U_H simulates the actions of M on input w
- If M halts, U_H accepts and halts (regardless of whether M accepts or rejects)
- But if M keeps running, U_H also keeps running

Halting Problem is not recursive

- We know that L_u is not recursive.
- Suppose, for contradiction, that the halting problem is recursive
- Then, if we wanted to determine whether a Turing machine M accepts w, we could do the following:
  - Determine whether M halts on input w. (We've just assumed that it is decidable)
  - If it does, execute M on w and see whether, when M halts, it does so in an accepting state. If so, accept (M, w)
  - If M does not halt on w, reject (M, w).

Halting Problem is not recursive

- Thus, we could test whether M accepts w by using our assumed algorithm for determining halting. We have "reduced L_u to the halting problem"
- So, if the halting problem is recursive, then so is L_u. But we know that L_u is not recursive, so the assumption that the halting problem was recursive must be false

Reductions

- If we have an algorithm to convert instances of a problem P_1 to instances of a problem P_2 so that the answer to P_1 can be deduced from the answer to P_2, then we say that P_1 *reduces* to P_2.
- We can use this to show that P_2 is at least as hard as P_1.
- If P_1 is not recursive, then P_2 cannot be recursive. If P_1 is non-RE, then P_2 cannot be RE.

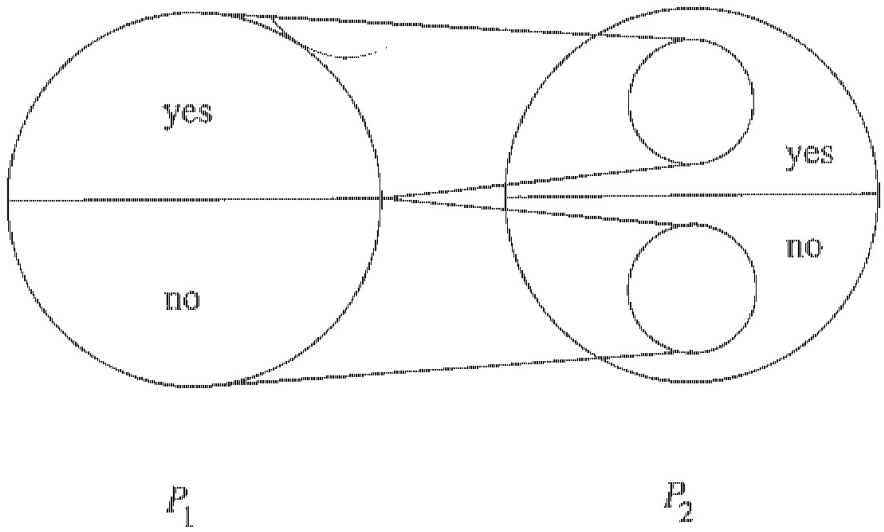


Figure 9.7: Reductions turn positive instances into positive, and negative to negative

Undecidable Problems About Turing Machines

- All problems about Turing machines that involve only the language that the TM accepts are undecidable
  i. Whether the language accepted by a TM is empty
  ii. Whether the language accepted by a TM is finite
  iii. Whether the language accepted by a TM is a regular language
  iv. Whether the language accepted by a TM is a context-free language

Other Undecidable Problems

Post's Correspondence Problem

- An instance of Post's Correspondence Problem (PCP) consists of two lists of strings over some alphabet Σ; the two lists must be of equal length
- We generally refer to the A and B lists, and write $A = w_1, w_2, \ldots, w_k$ and $B = x_1, x_2, \ldots, x_k$ for some integer k
- For each i, the pair (w_i, x_i) is said to be a corresponding pair

Other Undecidable Problems

Post's Correspondence Problem

- We say this instance of PCP has a solution, if there is a sequence of one or more integers $i_1, i_2, \ldots, i_m$ that, when interpreted as indexes for strings in the A and B lists, yield the same string
- That is, $w_{i_1} w_{i_2} \ldots w_{i_m} = x_{i_1} x_{i_2} \ldots x_{i_m}$ We say the sequence $i_1, i_2, \ldots, i_m$ is a solution to this instance of PCP

Example

	List A	List B
i	w_i	x_i
1	1	111
2	10111	10
3	10	0

- This PCP instance has a solution
- Let $m = 4$ and $i_1 = 2, i_2 = 1, i_3 = 1, i_4 = 3$
- $w_2 w_1 w_1 w_3 = x_2 x_1 x_1 x_3 = 101111110$
- 10111 1 1 10 = 10 111 111 0

Other Undecidable Problems

Theorem

Post's Correspondence Problem is undecidable

Other Undecidable Problems

- It is undecidable whether a CFG is ambiguous.
- Let G_1 and G_2 be context-free grammars, and let R be a regular expression. Then the following are undecidable:
  i. Whether the languages defined by G_1 and G_2 are same
  ii. Whether the language defined by G_1 and R are same
  iii. Whether the languages defined by G_1 is a subset of the language defined by G_2
  iv. Whether the languages defined by R is a subset of the language defined by G_1